

Aula 03 – Chamadas ao Sistema e Interrupções

Norton Trevisan Roman

12 de agosto de 2025

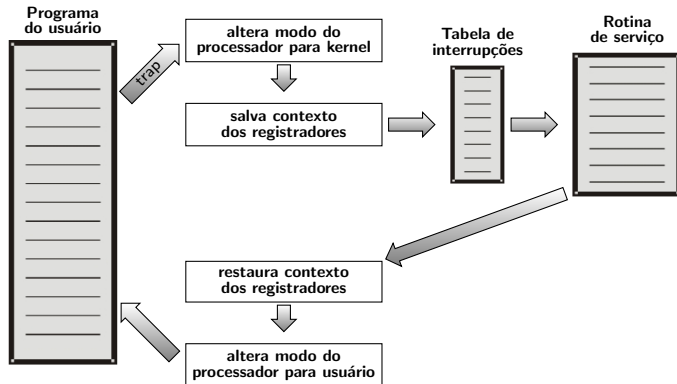
Chamadas ao Sistema

- Se uma aplicação precisa realizar alguma instrução privilegiada, ela realiza uma chamada de sistema
 - Esta altera do modo usuário para o modo kernel
 - Ex: Ler um arquivo

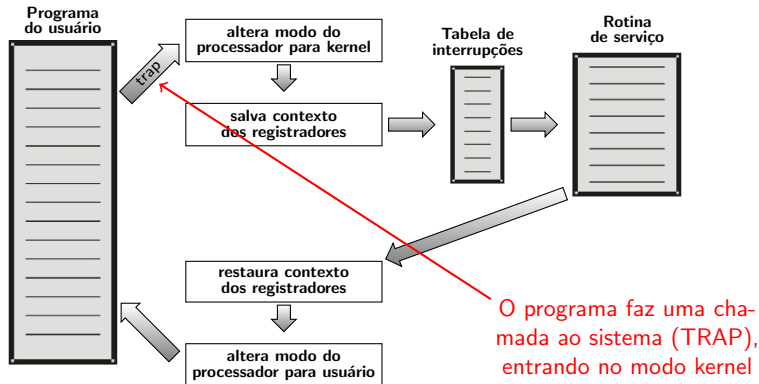
Chamadas ao Sistema

- Se uma aplicação precisa realizar alguma instrução privilegiada, ela realiza uma chamada de sistema
 - Esta altera do modo usuário para o modo kernel
 - Ex: Ler um arquivo
- Chamadas de sistemas são a porta de entrada para o modo Kernel;
 - São a interface entre os programas do usuário no modo usuário e o Sistema Operacional no modo kernel;
 - As chamadas diferem de SO para SO. No entanto, os conceitos relacionados às chamadas são similares independentemente do SO;

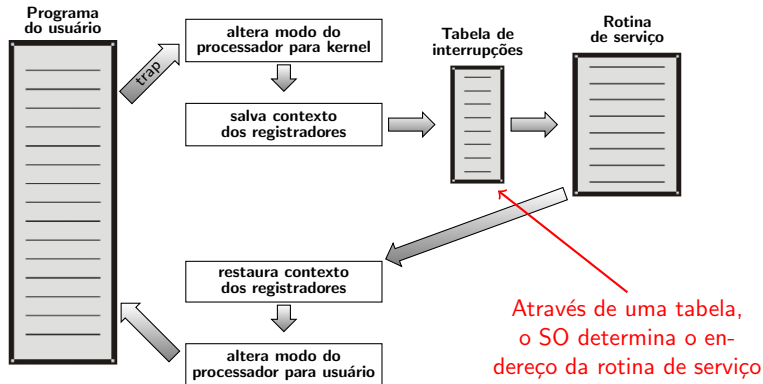
Chamadas ao Sistema



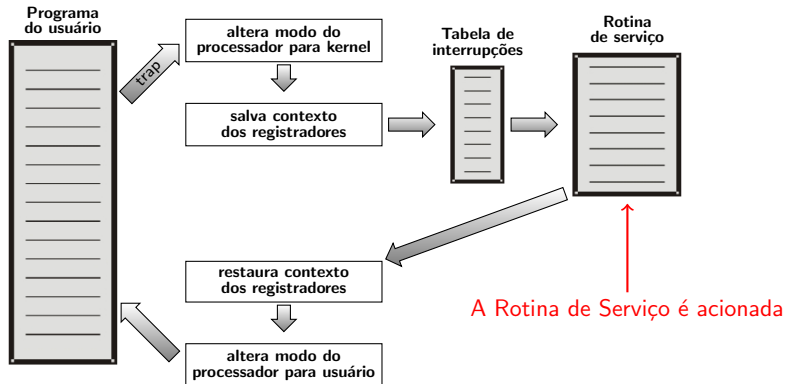
Chamadas ao Sistema



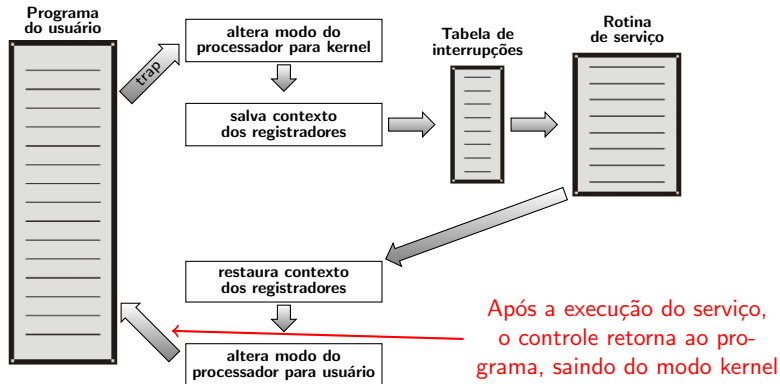
Chamadas ao Sistema



Chamadas ao Sistema



Chamadas ao Sistema



Chamadas ao Sistema

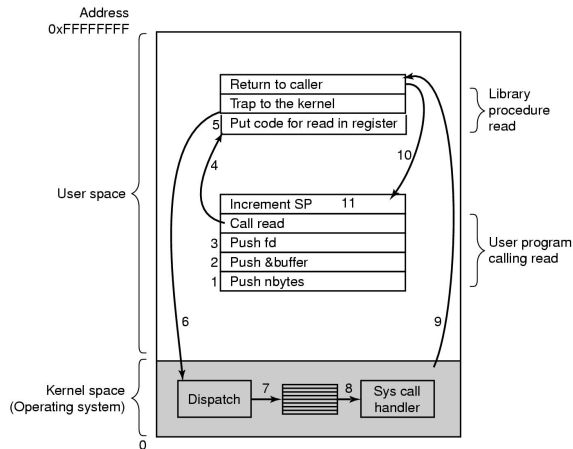
- O salvamento de contexto pode ser feito pela rotina de serviço
 - Caso em que ela salva somente os registradores que irá modificar
 - Antes de usar algum registrador, ela guarda seu valor anterior
 - Ao final, devolve esse valor
 - Com isso, ganha tempo, sem precisar transferir todos à memória

Chamadas ao Sistema

- O salvamento de contexto pode ser feito pela rotina de serviço
 - Caso em que ela salva somente os registradores que irá modificar
 - Antes de usar algum registrador, ela guarda seu valor anterior
 - Ao final, devolve esse valor
 - Com isso, ganha tempo, sem precisar transferir todos à memória
- Onde salva?
 - Depende do hardware
 - Já já veremos

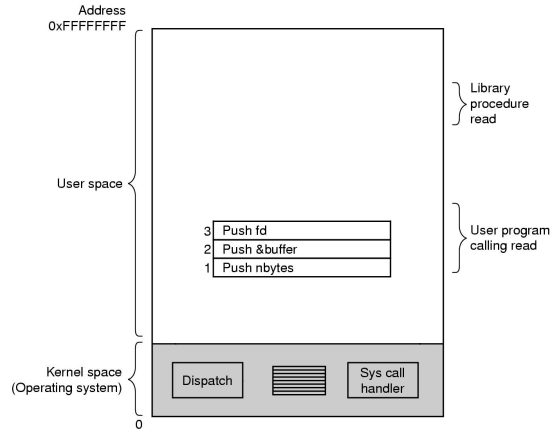
Chamadas ao Sistema – Exemplo

- `read(fd,buffer,nbytes)`
 - `fd`: especificador do arquivo
 - `buffer`: endereço na memória do primeiro byte da área onde deve ser armazenado o conteúdo lido
 - `nbytes`: número de bytes a serem lidos



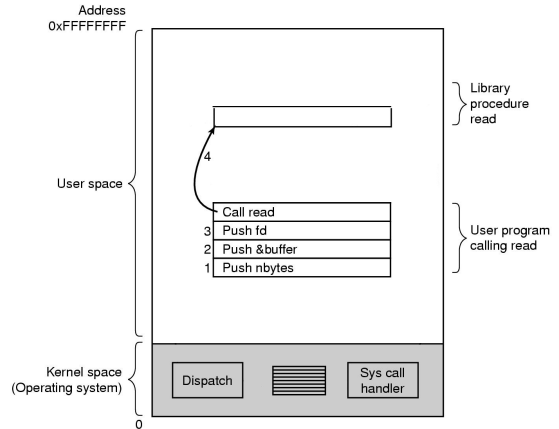
Exemplo: `read(fd,buffer,nbytes)`

- Chamando `read`...
- (1-3) O programa armazena os parâmetros na pilha de execução



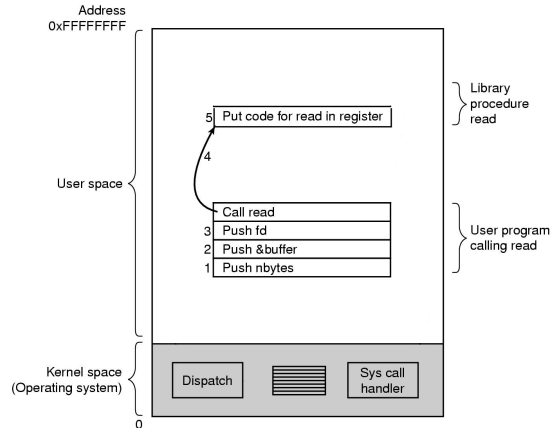
Exemplo: `read(fd,buffer,nbytes)`

- Chamando `read`...
- (1-3) O programa armazena os parâmetros na pilha de execução
- (4) Chama o procedimento `read` da biblioteca



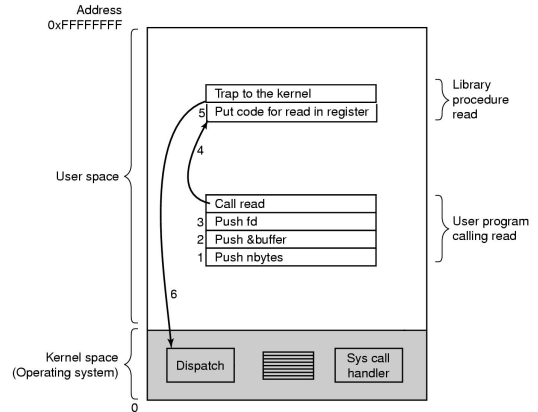
Exemplo: `read(fd,buffer,nbytes)`

- Chamando `read`...
 - (1-3) O programa armazena os parâmetros na pilha de execução
 - (4) Chama o procedimento `read` da biblioteca
- Executando `read`...
 - (5) `Read` coloca o número da chamada ao SO em um registrador específico



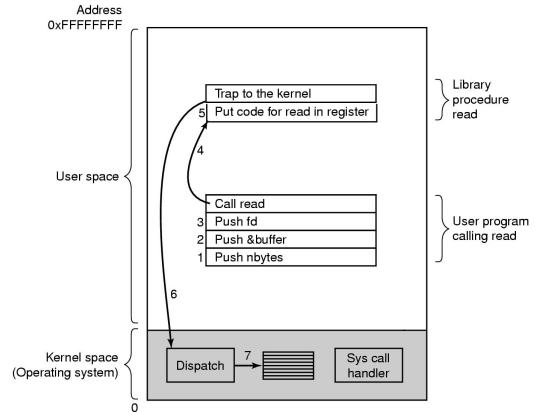
Exemplo: `read(fd,buffer,nbytes)`

- Executando read...
 - (6) Read executa uma instrução TRAP (passa do modo Usuário para Kernel e executa um determinado endereço no kernel → o S.O. tem o controle)



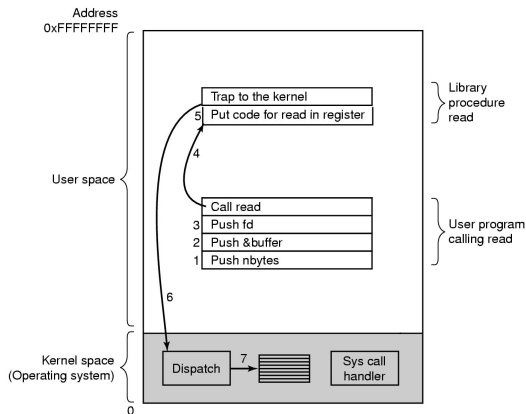
Exemplo: `read(fd,buffer,nbytes)`

- Executando read...
- (6) Read executa uma instrução TRAP (passa do modo Usuário para Kernel e executa um determinado endereço no kernel → o S.O. tem o controle)
- (7) O kernel (dispatcher) busca os parâmetros, verificando o número da chamada ao SO e chamando a rotina para seu tratamento



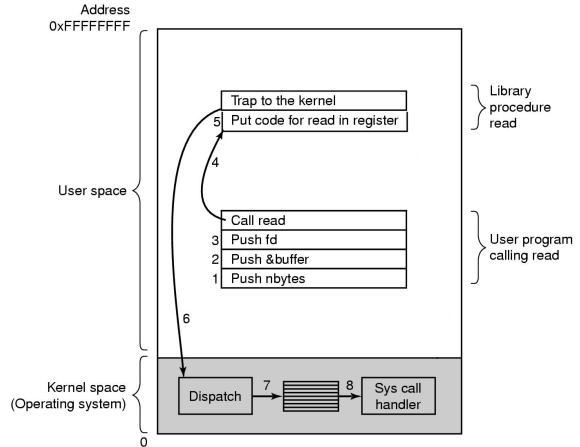
Exemplo: `read(fd,buffer,nbytes)`

- Executando read...
- (6) Read executa uma instrução TRAP (passa do modo Usuário para Kernel e executa um determinado endereço no kernel → o S.O. tem o controle)
- (7) O kernel (dispatcher) busca os parâmetros, verificando o número da chamada ao SO e chamando a rotina para seu tratamento
- Faz isso indexando uma tabela que contém na linha k um ponteiro para a rotina de serviço que executa a chamada de sistema k



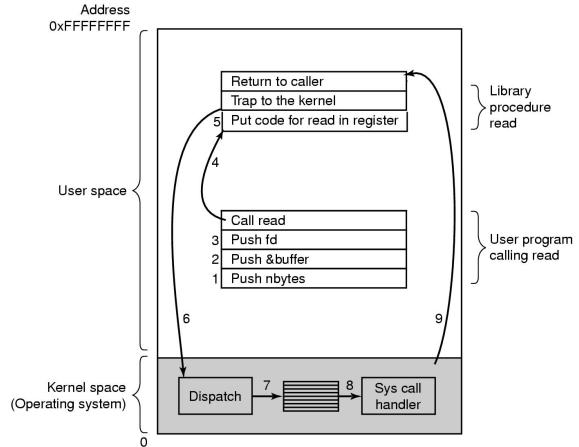
Exemplo: `read(fd,buffer,nbytes)`

- Executando `read`...
- (8) Essa rotina de serviço da chamada é executada (geralmente salvando o contexto)



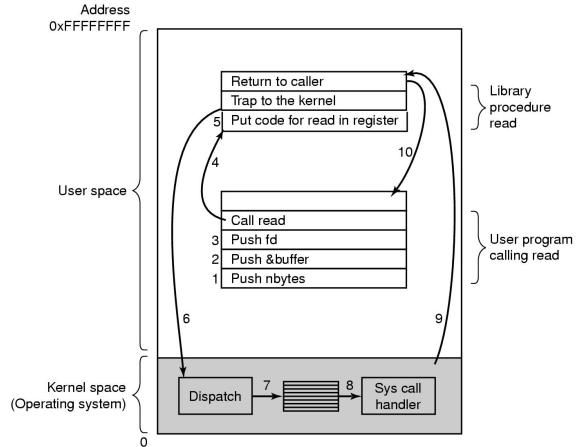
Exemplo: `read(fd,buffer,nbytes)`

- Executando `read`...
- (8) Essa rotina de serviço da chamada é executada (geralmente salvando o contexto)
- (9) Terminada a rotina, o contexto é restaurado, rodando o dispatcher. O controle pode então retornar à instrução seguinte à `TRAP`



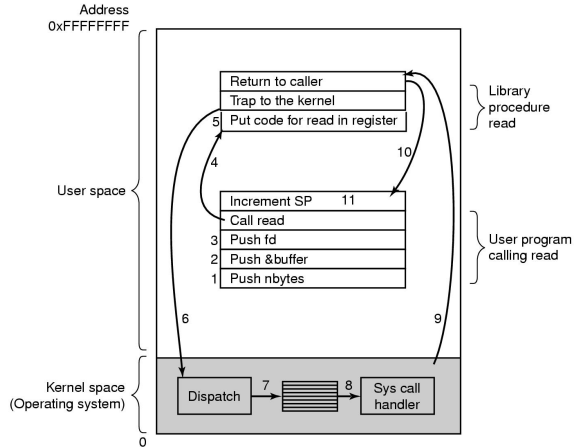
Exemplo: read(fd,buffer,nbytes)

- Executando read...
 - (8) Essa rotina de serviço da chamada é executada (geralmente salvando o contexto)
 - (9) Terminada a rotina, o contexto é restaurado, rodando o dispatcher. O controle pode então retornar à instrução seguinte à TRAP
 - (10) Read então retorna ao programa do usuário



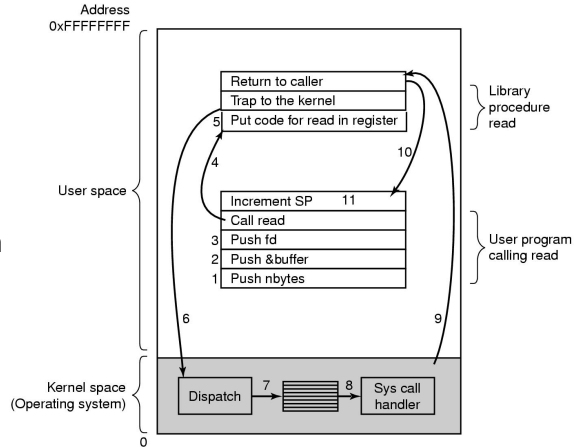
Exemplo: `read(fd,buffer,nbytes)`

- O programa limpa a pilha após a chamada do procedimento (11)
- Incrementa o ponteiro da pilha o suficiente para remover os parâmetros da chamada a `read` (lembre que a pilha está de cabeça para baixo para baixo)



Exemplo: `read(fd,buffer,nbytes)`

- Voltemos ao passo 9
 - Em vez de retornar, a chamada pode bloquear quem a chamou
 - Ex: esperando do teclado
 - O S.O. nesse caso verifica se algum outro processo pode ser executado
 - Quando a entrada estiver disponível, os passos 9 a 11 desse processo podem ser executados



Chamadas ao Sistema – Invocação direta

- write() e exit() através da interrupção 0x80 (trap Linux para x86)

```
section    .data                                ;declaração da seção
    msg    db    "Ola, mundo!",0xa             ;declara localização (declare byte): nosso string
    len    equ    $ - msg                       ;define constante (equ): tamanho do nosso string
section    .text                                ;declaração de seção
    global _start                               ;ponto de entrada para o linker (ld)
```

```
_start:                                         ;diz ao linker o ponto de entrada
    ;escreve o string na stdout
    mov     edx,len                            ;terceiro argumento: tamanho da mensagem
    mov     ecx,msg                            ;segundo argumento: ponteiro para a mensagem a ser escrita
    mov     ebx,1                              ;primeiro argumento: descritor de arquivo (stdout)
    mov     eax,4                              ;número da chamada ao sistema (sys_write)
    int     0x80                               ;chama o kernel

    ;e sai
    mov     ebx,0                              ;primeiro argumento da chamada ao sistema: código de saída
    mov     eax,1                              ;número da chamada ao sistema (sys_exit)
    int     0x80                               ;chama o kernel
```


Invocação direta – Pé no Hardware

- O que acontece que esquecemos de chamar `exit()` e o programa ainda detém tempo de CPU?

```
;e sai
mov     ebx,0      ;primeiro argumento da chamada ao sistema: código de saída
mov     eax,1      ;número da chamada ao sistema (sys_exit)
int     0x80       ;chama o kernel
```

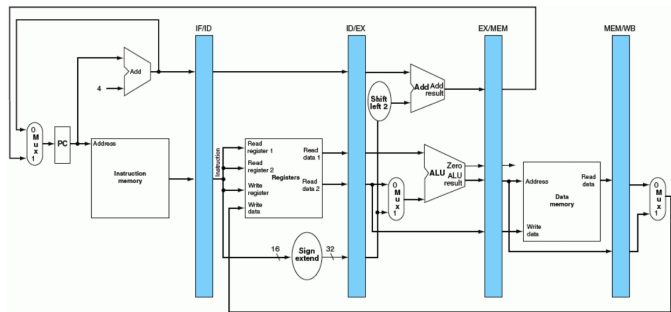
Invocação direta – Pé no Hardware

- O que acontece que esquecemos de chamar `exit()` e o programa ainda detém tempo de CPU?

;e sai

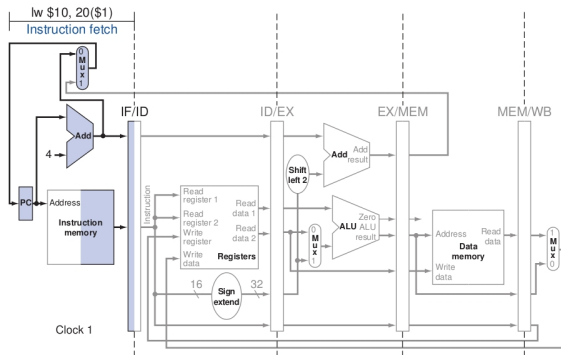
```
mov    ebx,0    ;primeiro argumento da chamada ao sistema: código de saída
mov    eax,1    ;número da chamada ao sistema (sys_exit)
int    0x80     ;chama o kernel
```

- Olhemos a pipeline (BEM simplificada):



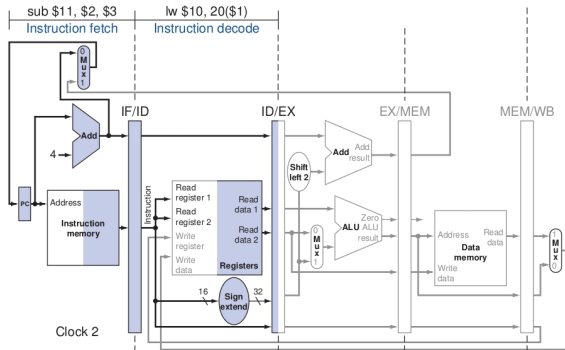
Invocação direta – Pé no Hardware

- Primeiro ciclo de clock
 - A instrução é buscada na memória
 - 4 (bytes $\rightarrow$ 32b) são somados ao PC e armazenados nele.



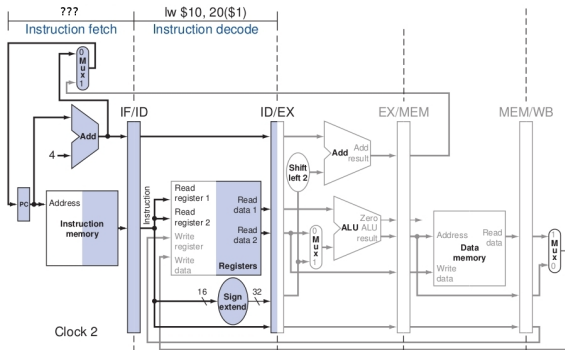
Invocação direta – Pé no Hardware

- Segundo ciclo de clock
 - Nova instrução é buscada na memória (enquanto a outra está no estágio 2)
 - 4 (bytes $\rightarrow$ 32b) são somados ao PC e armazenados nele.



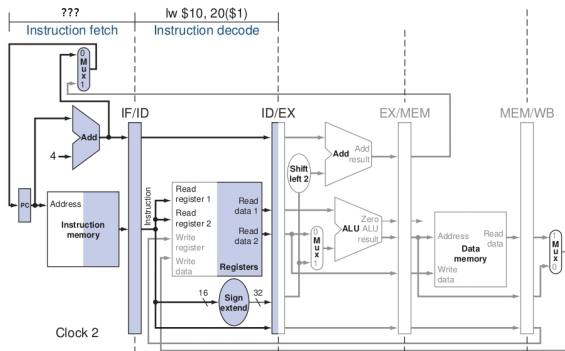
Invocação direta – Pé no Hardware

- Segundo ciclo de clock
 - Mas se o programa havia acabado, que instrução é essa?



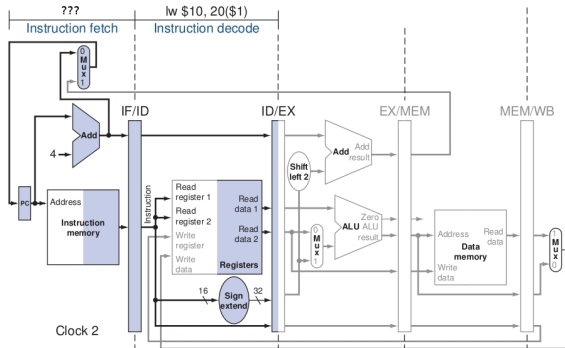
Invocação direta – Pé no Hardware

- Segundo ciclo de clock
 - Mas se o programa havia acabado, que instrução é essa?
 - O que há na memória naquela posição



Invocação direta – Pé no Hardware

- Segundo ciclo de clock
 - A instrução pode ser algo que não deveríamos fazer (pouco provável)
 - Ou instrução desconhecida → instrução ilegal



Invocação direta – exit()

- Esquecer de chamá-lo pode fazer com que:
 - Uma instrução ilegal seja rodada → o programa é forçosamente parado
 - Uma instrução legal, porém indesejada, seja rodada → comportamento imprevisível

Invocação direta – `exit()`

- Esquecer de chamá-lo pode fazer com que:
 - Uma instrução ilegal seja rodada → o programa é forçosamente parado
 - Uma instrução legal, porém indesejada, seja rodada → comportamento imprevisível
- Por conta disso compiladores sempre incluem um `exit()` ao final do código compilado, se o programador não o fizer explicitamente

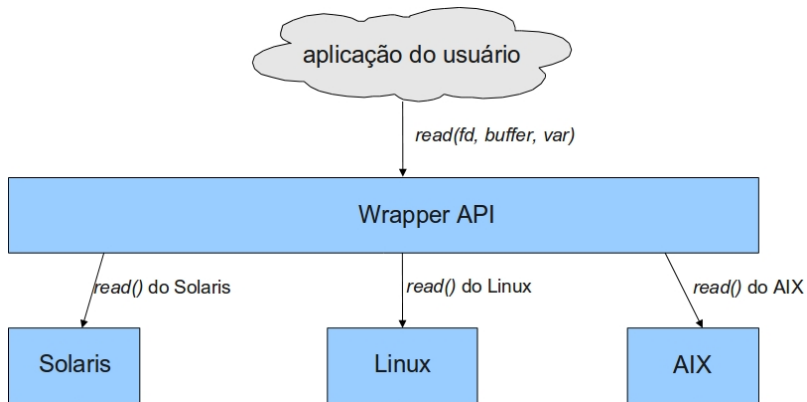
Chamadas ao Sistema

Interface das Chamadas ao Sistema (Wrappers)

- Chamadas a bibliotecas
 - Em Windows, desacopladas das chamadas reais ao sistema
- Interface de programação fornecida pelo SO
- Geralmente escrita em linguagem de alto nível (C, C++ ou Java)
- Normalmente as aplicações utilizam uma Application Program Interface (API)
 - Interface que encapsula o acesso direto às chamadas ao sistema

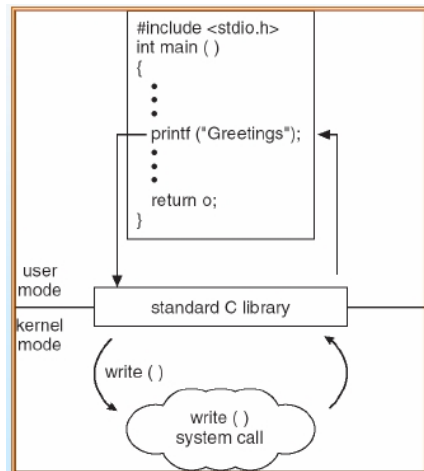
Chamadas ao Sistema

- Portabilidade usando Wrappers



Chamadas ao Sistema

- Programa em C que invoca a função de biblioteca printf(), que por sua vez chama o system call write()
- write() e exit() através da instrução int 0x80



Chamadas ao Sistema

Interface das Chamadas de Sistema (Wrappers)

- Mais utilizadas:
 - Win32 API para Windows
 - POSIX API para praticamente todas as versões de UNIX
 - Java API para a Java Virtual Machine (JVM).
- Motivos para usar APIs em vez das chamadas diretas ao sistema
 - Portabilidade – independência da plataforma
 - Esconder complexidade inerente às chamadas ao sistema
 - Acréscimo de funcionalidades que otimizam o desempenho

Interrupções

- Vimos que um software pode interromper seu próprio processo (com uma chamada ao sistema)
 - Via traps (Interrupções de software ou Exceções)
 - Para isso, o software tem que estar rodando

Interrupções

- Vimos que um software pode interromper seu próprio processo (com uma chamada ao sistema)
 - Via traps (Interrupções de software ou Exceções)
 - Para isso, o software tem que estar rodando
- Mas como pode o escalonador interromper um determinado processo se ele mesmo não está rodando?

Interrupções

- Vimos que um software pode interromper seu próprio processo (com uma chamada ao sistema)
 - Via traps (Interrupções de software ou Exceções)
 - Para isso, o software tem que estar rodando
- Mas como pode o escalonador interromper um determinado processo se ele mesmo não está rodando?
- Via interrupções de hardware
 - Sinal elétrico no Hardware
 - Causa: dispositivos de E/S ou o clock

Interrupções – Tratamento

- Quando o SO inicia, ele instala um tratador de interrupções em um endereço específico da memória, definido pelo hardware
 - Ex: mips → 0x80000180
- Cria também um Arranjo de Interrupções (também chamado de arranjo de traps)
- Ex: primeiras 256 palavras do segmento de código do SO (x8000-x80FF)
 - Cada palavra é um jump (JMP) para uma determinada rotina
 - Ou para uma rotina especial, se o código da interrupção não estiver nesses 256, caso em que essa rotina mata o processo

Interrupções – Tratamento

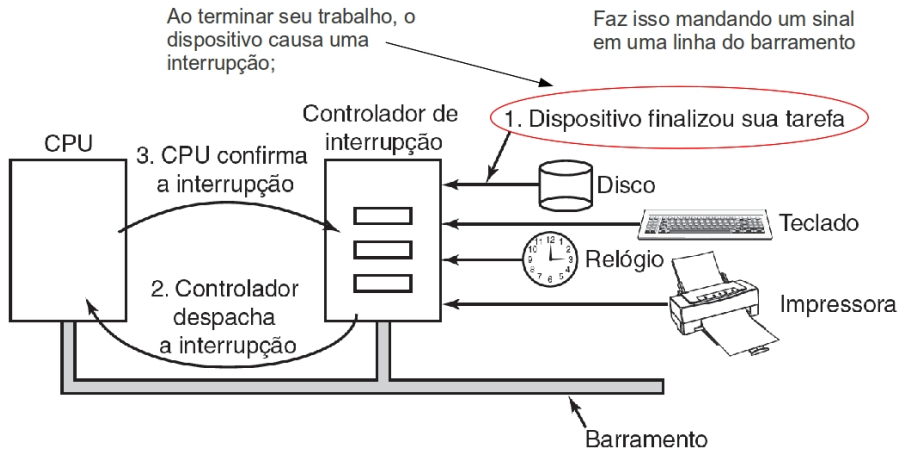
Arranjo de Interrupções

- Locação de memória (geralmente próxima da parte mais baixa da memória), associada a cada classe de dispositivos de E/S (inclusive clock)
 - Cada classe (disco, clock etc) pode ter um arranjo diferente
- Usado para gerenciar interrupções dos programas
 - Contém os endereços dos procedimentos dos serviços de interrupção
 - Junto com o BCP (mais adiante...), controla a execução dos processos

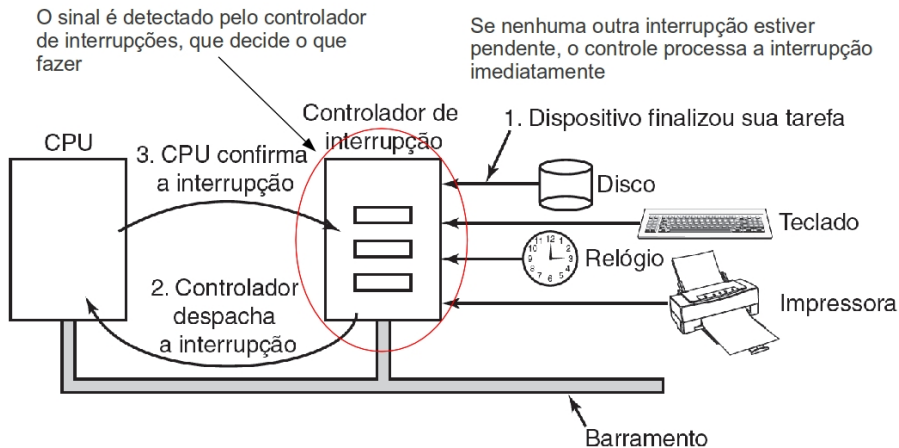
Interrupções × Traps

- Interrupções:
 - Evento externo ao processador
 - Gerados por dispositivos que precisam da atenção do SO
 - Podem não estar relacionadas ao processo que está rodando
- Traps:
 - Evento inesperado vindo de dentro do processador
 - Causadas pelo processo corrente no processador (seja por chamada ao SO, seja por instrução ilegal)

Interrupção – Pé no Hardware



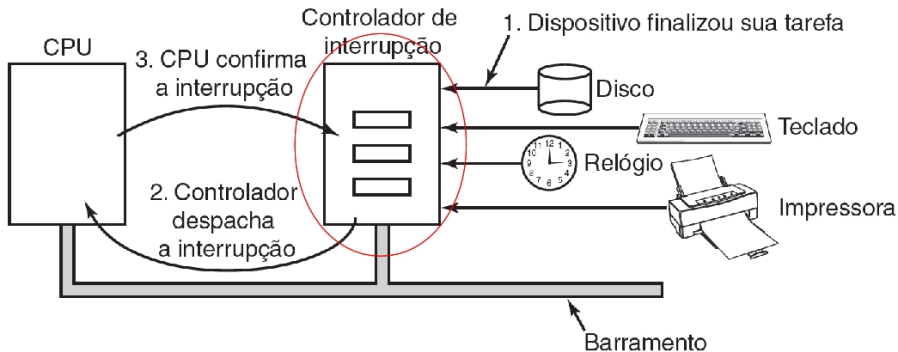
Interrupção – Pé no Hardware



Interrupção – Pé no Hardware

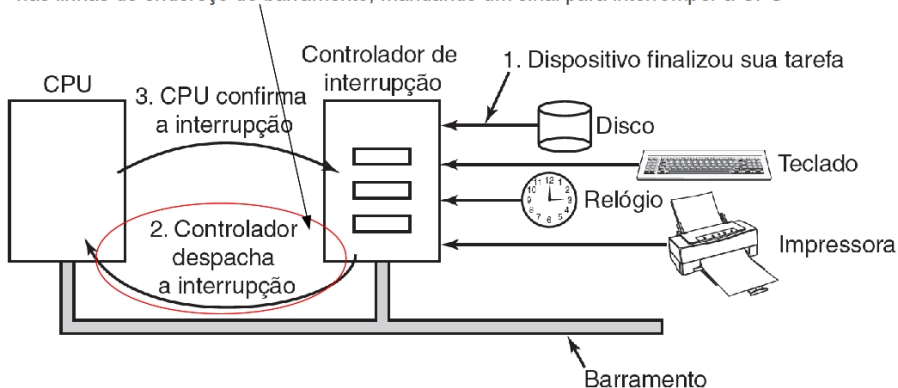
Se alguma outra estiver em progresso, ou outro dispositivo fez um pedido simultâneo, em uma linha de interrupção de maior prioridade no barramento, o dispositivo é ignorado.

Caso em que continua a mandar o sinal de interrupção ao barramento



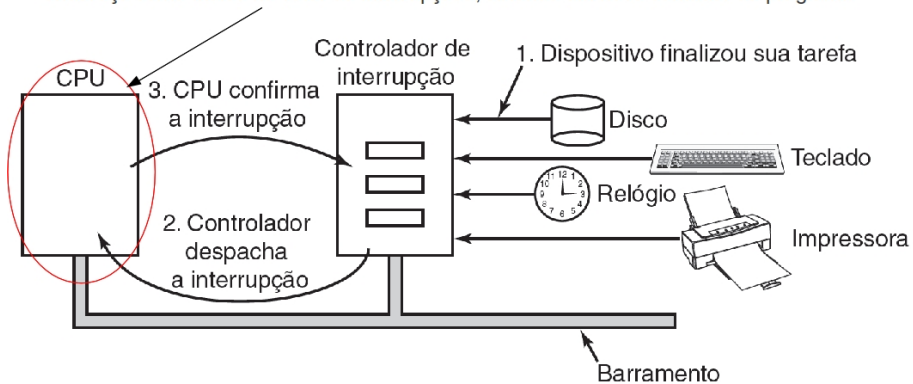
Interrupção – Pé no Hardware

Para tratar a interrupção, o controlador coloca um número, especificando o dispositivo, nas linhas de endereço do barramento, mandando um sinal para interromper a CPU



Interrupção – Pé no Hardware

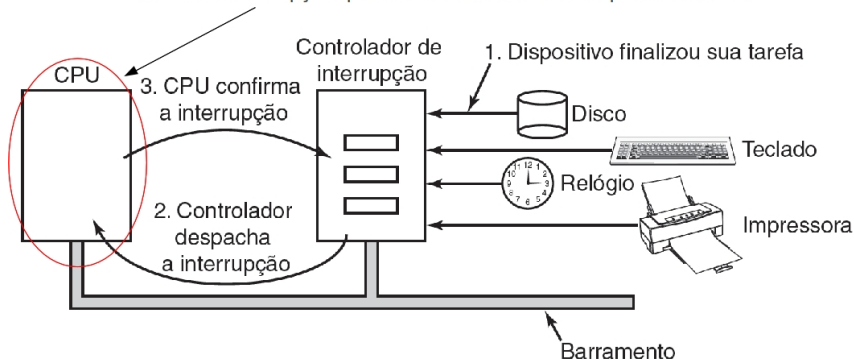
O sinal faz com que a CPU pare o que está fazendo. Ela então usa o número nas linhas de endereço como índice no vetor de interrupções, obtendo um novo contador de programa



Interrupção – Pé no Hardware

O novo contador de programa aponta para o início do procedimento de tratamento da interrupção.

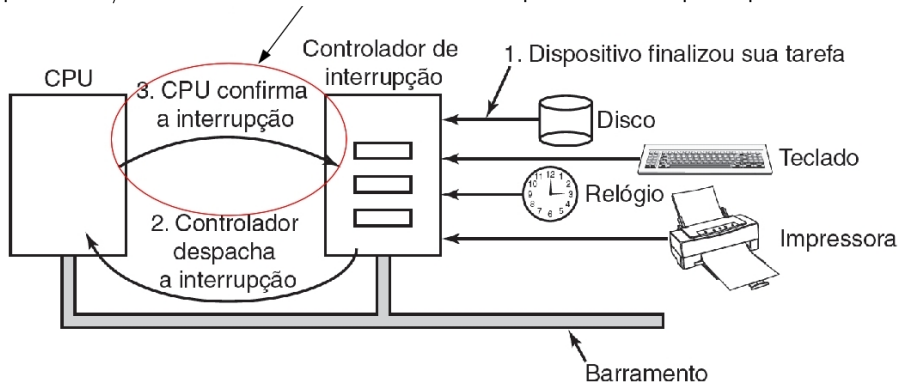
Traps e interrupções usam o mesmo mecanismo a partir desse ponto
O vetor de interrupções pode estar tanto no hardware quanto na memória



Caso na memória, haverá um registrador da CPU (carregado pelo SO), que aponta para sua origem

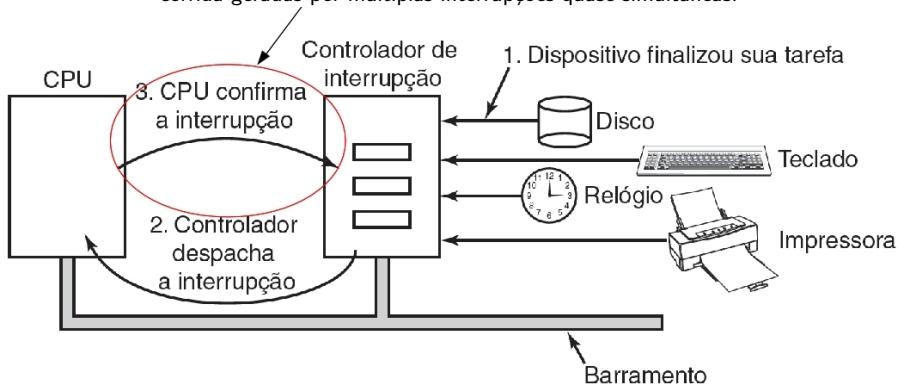
Interrupção – Pé no Hardware

Pouco depois de começar a execução, a rotina de tratamento confirma a interrupção, escrevendo um certo valor em uma das portas de E/S do controlador. Isso diz ao controlador que ele está livre para repassar outra interrupção.



Interrupção – Pé no Hardware

Assim, a CPU atrasa essa confirmação até estar livre para lidar com a próxima interrupção, evitando assim condições de corrida geradas por múltiplas interrupções quase simultâneas.



Interrupção

- Informação salva (temporariamente) pelo hardware antes de iniciar a rotina de serviço:
 - Pelo menos, o contador de programa
 - Pois será sobrescrito pelo da rotina de tratamento
 - Em último caso, todos os registradores gerais e alguns internos
- Onde salvar? Não há solução perfeita.
 - Registradores internos
 - Problema: Não há como enviar a confirmação (passo 3) ao controlador de interrupções até que toda a informação neles tenha sido usada (evitando sobrescrita) → toma tempo

Interrupção

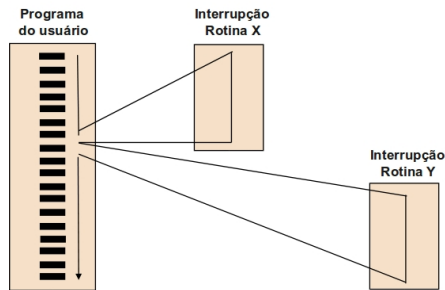
- Pilha (do processo do usuário)
 - O ponteiro da pilha pode não ser legal (por escalonamento, página não na memória, por exemplo)
 - Poderia ser o final de uma página, levando a um page fault durante a interrupção (e onde salvar o estado para tratar a page fault?)
- Pilha do kernel
 - Maior chance do ponteiro ser legal
 - O chaveamento para modo núcleo pode exigir mudança de contexto, pela MMU, podendo mudar cache e TLB → toma tempo (veremos mais adiante)
- Ex: MIPS → em uma região da memória, cujo endereço é armazenado em um registrador específico

Interrupção

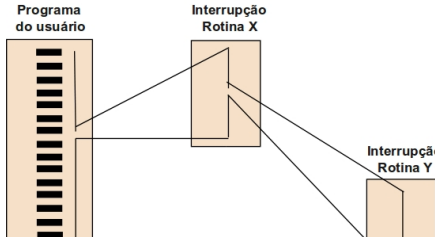
- O sinal (linha) de interrupção é exibido dentro de cada ciclo de instrução do processador
- A cada ciclo de instrução, a CPU:
 - Verifica se existe interrupção. Se não, busca a próxima instrução
 - Se existir interrupção pendente:
 - Suspende a execução do programa
 - Salva contexto
 - Atualiza PC (Program Counter) → PC aponta para a ISR (rotina de atendimento de interrupção)
 - Executa a rotina de tratamento da interrupção
 - Recarrega contexto e continua processo interrompido

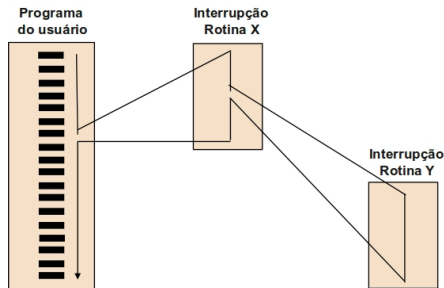
Múltiplas Interrupções

- Modelo seqüencial
 - O tratador de interrupções desabilita as interrupções
 - Uma nova interrupção só é tratada após o retorno da anterior
 - A interrupção pode demorar a ser tratada, o que pode eventualmente ocasionar uma perda de dados
- Finalizado o tratador de interrupção, o processador checa por interrupções adicionais



Múltiplas Interrupções

- Modelo cascata
 - Interrupções têm prioridade
 - Interrupções com alta prioridade interrompem rotinas de serviço de interrupções de menor prioridade
 - Exemplos de prioridade:
 - impressora (menor)
 - disco
 - comunicação (maior)
- 
- O diagrama ilustra o modelo cascata de interrupções. À esquerda, um retângulo alto e estreito rotulado 'Programa do usuário' contém uma barra vertical com dez barras horizontais pretas. À direita, há dois retângulos menores rotulados 'Interrupção Rotina X' e 'Interrupção Rotina Y'. Linhas de conexão mostram a sequência de execução: uma linha desce da barra de interrupções no programa do usuário até o topo de 'Interrupção Rotina X'; uma linha desce do topo de 'Interrupção Rotina X' até o topo de 'Interrupção Rotina Y'; e uma linha desce do topo de 'Interrupção Rotina Y' de volta à barra de interrupções no programa do usuário, completando o ciclo.



Referências Adicionais

- Patterson, D.A.; Hennessy, J.L: Computer Organization and Design: The Hardware Software interface. 3 ed. Elsevier:Nova Iorque, 2005.
- <http://www.codinghorror.com/blog/2008/01/\understanding-user-and-kernel-mode.html>
- <http://www.cs.virginia.edu/~evans/cs216/guides/x86.html>
- <http://www.c-jump.com/CIS77/ASM/Assembly/lecture.html>
- <http://www.csee.umbc.edu/~chang/cs313.s02/stack.shtml>